

AFRILANG Stdlib

std/m/listchunk

Français. Bibliothèque AFRILANG 'std/m/listchunk' (niveau medium, catégorie medium). Elle expose 5 fonction(s) : chunkCount, take, drop, slice, chunkAt.

```
'import "std/m/listchunk"' · 'use listchunk'
```

```
- 'chunkCount(v list number, size number) → number' - 'take(v list number, n number) → list number' - 'drop(v list number, n number) → list number' - 'slice(v list number, start number, end number) → list number' - 'chunkAt(v list number, index number, size number) → list number'
```

English. AFRILANG library 'std/m/listchunk' (tier medium, category medium). Exports 5 function(s): chunkCount, take, drop, slice, chunkAt.

```
'import "std/m/listchunk"' · 'use listchunk'
```

```
- 'chunkCount(v list number, size number) → number' - 'take(v list number, n number) → list number' - 'drop(v list number, n number) → list number' - 'slice(v list number, start number, end number) → list number' - 'chunkAt(v list number, index number, size number) → list number'
```

Catégorie / Category Modules medium (m/)

Niveau / Tier medium

Fonctions / Functions 5

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/m/listchunk' (niveau medium, catégorie medium). Elle expose 5 fonction(s) : chunkCount, take, drop, slice, chunkAt.

'import "std/m/listchunk"' · 'use listchunk'

```
- `chunkCount(v list number, size number) → number`
- `take(v list number, n number) → list number`
- `drop(v list number, n number) → list number`
- `slice(v list number, start number, end number) → list number`
- `chunkAt(v list number, index number, size number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/m/listchunk"
use listchunk
```

Niveau : medium · Catégorie : Modules medium (m/)
Bibliothèques intermédiaires – import std/m.../

3. Référence API

- chunkCount(v list number, size number) → number•
- take(v list number, n number) → list•
- drop(v list number, n number) → list•
- slice(v list number, start number, end number) → list•
- chunkAt(v list number, index number, size number) → list

4. Exemples d'utilisation

```
import "std/m/listchunk"
use listchunk
```

```
create result = chunkCount(/* args */)
say result
```

```
create result = take(/* args */)
say result
```

```
create result = drop(/* args */)
say result
```

```
create result = slice(/* args */)
say result
```

```
create result = chunkAt(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/m/listchunk"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules `stdlib` de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module `listchunk`

```
function chunkCount(v list number, size number) returns number  
end
```

```
function take(v list number, n number) returns list number  
end
```

```
function drop(v list number, n number) returns list number  
end
```

```
function slice(v list number, start number, end number) returns list number  
end
```

```
function chunkAt(v list number, index number, size number) returns list number  
end  
end
```

English

1. Overview

AFRILANG library 'std/m/listchunk' (tier medium, category medium). Exports 5 function(s): chunkCount, take, drop, slice, chunkAt.

'import "std/m/listchunk"' · 'use listchunk'

```
- `chunkCount(v list number, size number) → number`
- `take(v list number, n number) → list number`
- `drop(v list number, n number) → list number`
- `slice(v list number, start number, end number) → list number`
- `chunkAt(v list number, index number, size number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/m/listchunk"
use listchunk
```

Tier: medium · Category: Medium modules (m/)
Intermediate libraries – import std/m.../

3. API reference

- chunkCount(v list number, size number) → number•
- take(v list number, n number) → list•
- drop(v list number, n number) → list•
- slice(v list number, start number, end number) → list•
- chunkAt(v list number, index number, size number) → list

4. Usage examples

```
import "std/m/listchunk"
use listchunk
```

```
create result = chunkCount(/* args */)
say result
```

```
create result = take(/* args */)
say result
```

```
create result = drop(/* args */)
say result
```

```
create result = slice(/* args */)
say result
```

```
create result = chunkAt(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/m/listchunk"`` at the top of your file.
- Run ``afri lang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module listchunk

```
function chunkCount(v list number, size number) returns number
end
```

```
function take(v list number, n number) returns list number
end
```

```
function drop(v list number, n number) returns list number
end
```

```
function slice(v list number, start number, end number) returns list number
end
```

```
function chunkAt(v list number, index number, size number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/m/listchunk"`
- Vérification : `afri lang check`
- Documentation : <https://afri lang.dev/stdlib/std/m/listchunk/>

Source (extrait)

```
module listchunk

function chunkCount(v list number, size number) returns number
end

function take(v list number, n number) returns list number
end

function drop(v list number, n number) returns list number
end

function slice(v list number, start number, end number) returns list number
end

function chunkAt(v list number, index number, size number) returns list number
end
end
```